

面向RustOS的可靠性增强内核架构设计

黑袍纠察队

国防科技大学



CONTENTS

- 绪论
- 可靠性增强内核架构设计
- 实验分析
- 总结展望



01

绪论

绪论

- 项目背景
- 项目设计目标
- 本项目工作



项目背景

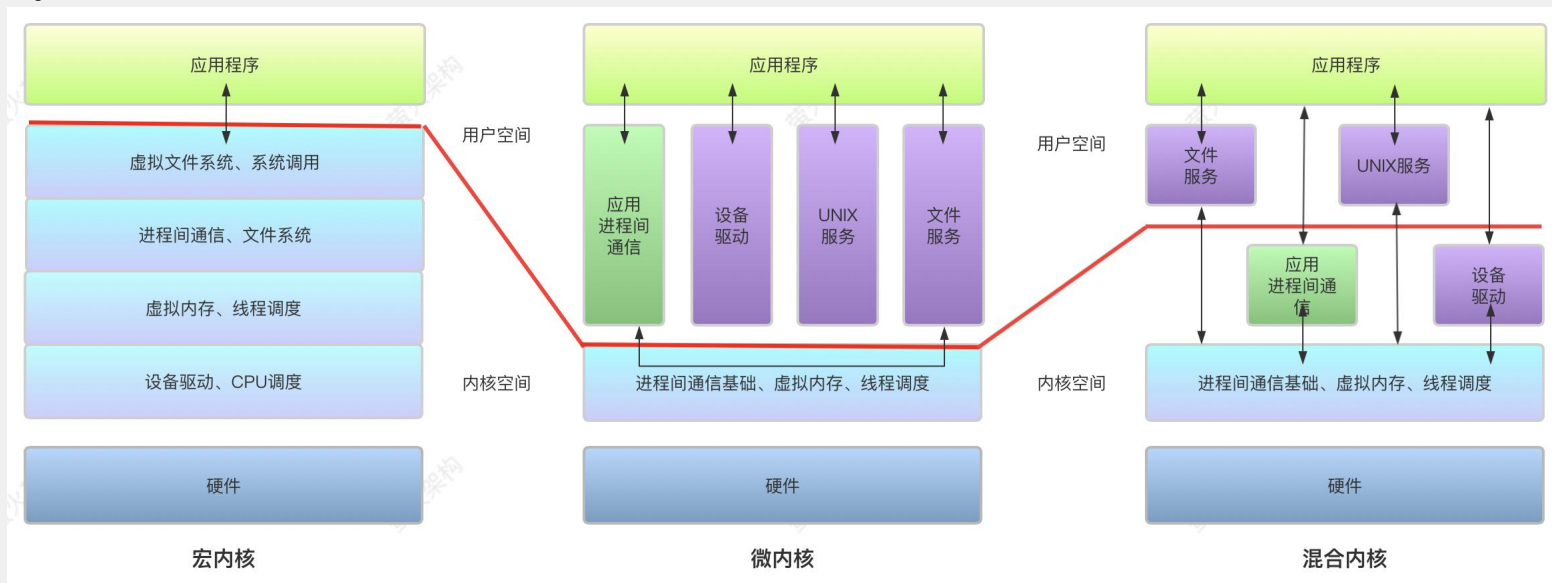
- 操作系统可靠性
- 主流内核架构
- 线程模型
- 基于Rust语言改进内核架构

项目背景：操作系统可靠性

- 可靠性是操作系统的重要设计目标
 - 1996 年欧洲空间局（ESA）阿丽亚娜 5 型火箭（Ariane 5）由于软件缺陷而爆炸。
- 良好的可靠性设计可以帮助操作系统持续为用户提供服务。
- 操作系统内核架构是影响操作系统可靠性的重要因素。

项目背景：主流内核架构

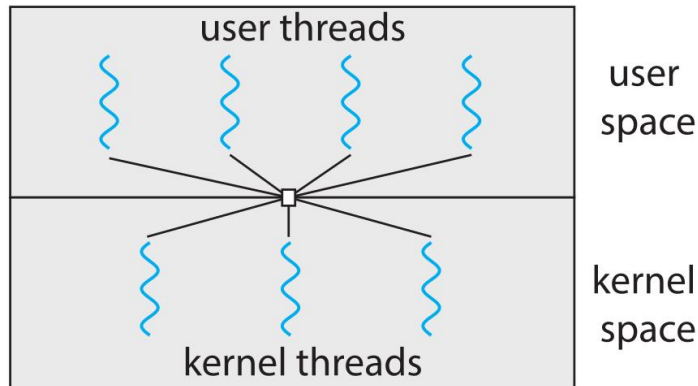
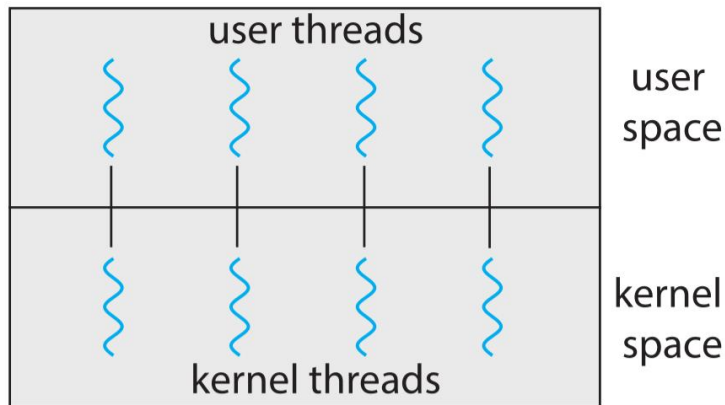
- 宏内核：所有内核服务全部集成在内核态，**性能好，可靠性、可拓展性差。**
- 微内核：核心服务在内核态，非核心服务用户态，**可靠性强，性能较差。**
- 混合内核：折中方案，非核心服务以**独立内核线程**的方式运行在内核态，**避免IPC开销。**



项目背景：线程模型

一对一线程模型：一个用户线程配备一个内核线程（有独立内核栈），内核需要**限制用户线程数量、内核线程数量多，上下文开销大。**

多对多线程模型：每个内核线程服务多个用户线程，**节省内存，节省上下文切换开销。**



项目背景：基于Rust语言改进内核架构

- 内存安全性
 - 在同一地址空间实现内存隔离
 - 优化微内核性能，用户态服务下沉内核态，由Rust保证访存隔离。
- 无栈异步协程支持
 - 支持一个线程中的多个异步任务并发
 - 优化线程模型，一个内核线程服务多个用户线程

绪论

- 研究背景
- 项目设计目标
- 本文工作

研究设计目标

- 基于Rust语言，设计实现一个多对多线程模型的混合内核，内存安全特性代替传统地址空间隔离,保证服务间隔离性的同时,避免IPC 产生的性能开销;
- 在第一点基础上,设计一种内核服务故障恢复机制,进一步提升内核可靠性,应对内存缺陷之外的其它缺陷(如逻辑错误触发 panic)。

绪论

- 研究背景
- 项目设计目标
- 本项目工作

本项目工作

- 设计实现了一个混合内核架构；
- 基于Rust无栈异步协程机制和内核服务线程模型实现了一个多对多线程模型；
- 为了进行对比，实现了宏内核、混合内核两种内核对象，内核可以以两种架构模式运行；
- 直接实现Linux系统调用；
- 初步完成了musl-libc支持，支持c语言应用程序（静态链接）；
- 正在逐步完成对sqlite3, busybox等典型Linux应用程序的支持；

本文工作

```
mangp@mangps:~/桌面/OS/mini-Rust-os$ make count
cloc . --exclude-dir=crates,target,musl,tutorial,sqlite3,busybox
  217 text files.
  207 unique files.
   37 files ignored.
```

github.com/AlDanial/cloc v 1.98 T=0.08 s (2643.9 files/s, 194015.2 lines/s)

Language	files	blank	comment	code
Rust	160	1256	2181	9270
Assembly	2	2	4	1312
C	11	54	32	264
TOML	22	28	8	248
Markdown	2	79	0	140
make	3	38	15	123
JSON	3	2	0	58
YAML	1	1	3	36
Linker Script	1	7	0	22
Text	2	3	0	4
SUM:	207	1470	2243	11477

代码目录树

项目的代码目录树如下

```
.
├── boot                // 启动内核
├── crates               // 有修改的第三方库
├── executor            // 协程执行器
├── hal                 // 硬件抽象层
├── hybrid-objects      // 混合内核对象
├── hybrid-syscalls    // 混合内核系统调用
├── loader              // 加载器
├── misc                // 杂项
├── monolithic-objects // 宏内核对象
├── monolithic-syscalls // 宏内核系统调用
├── musl                // musl源码
├── Ncore               // 顶层内核
├── rcore-fs-use        // 构建文件系统
├── tutorial            // 文档
├── user-c              // C语言用户程序
├── user-components    // 用户态组件
├── user-rs             // Rust用户程序
└── Question.md        // 本题目简介
```



02

可靠性增强内核架构设计

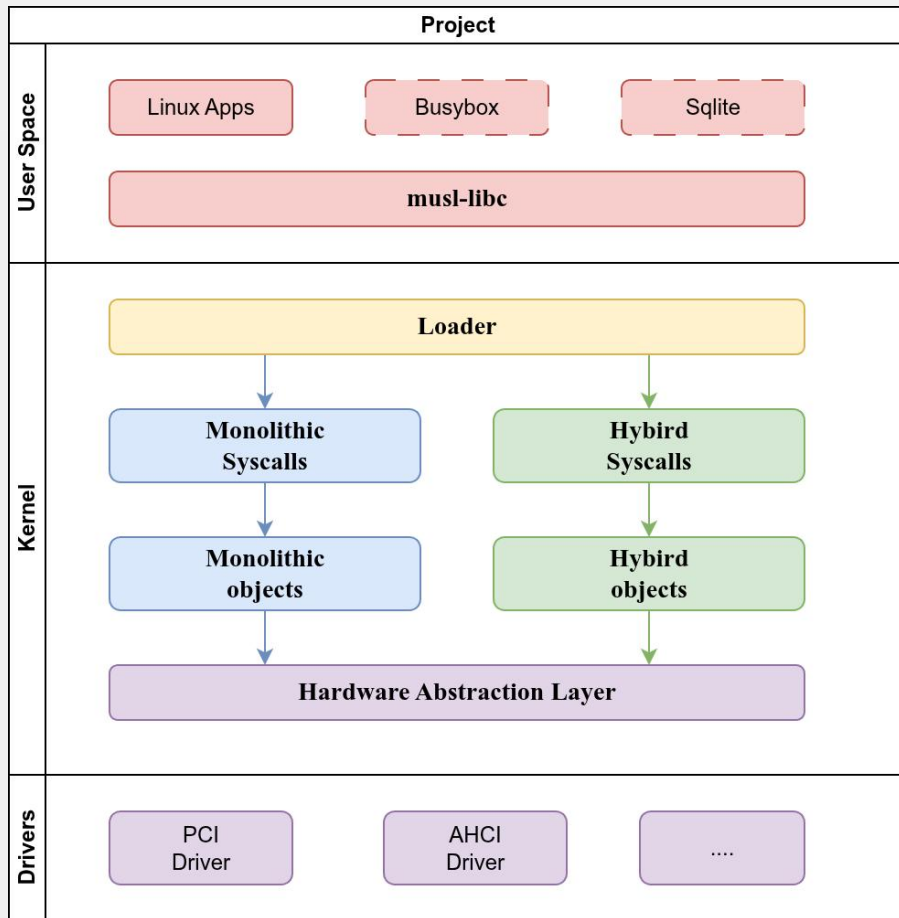


可靠性增强内核架构设计

- 项目总体架构
- 多对多线程模型的混合内核架构
- 内核服务线程故障恢复

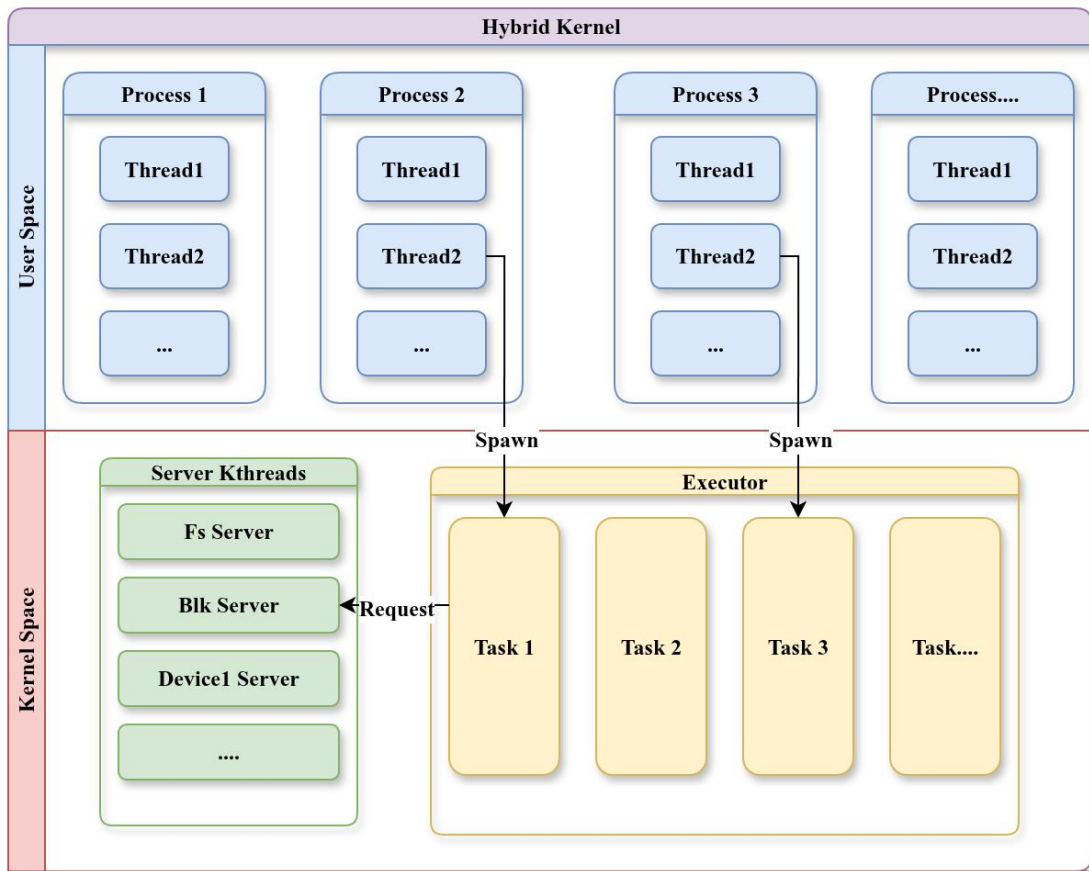
可靠性增强内核架构设计：项目总体架构

- 编译时利用Rust feature选择宏内核或混合内核
- 直接提供Linux系统调用
- 初步支持musl-libc和C语言应用程序



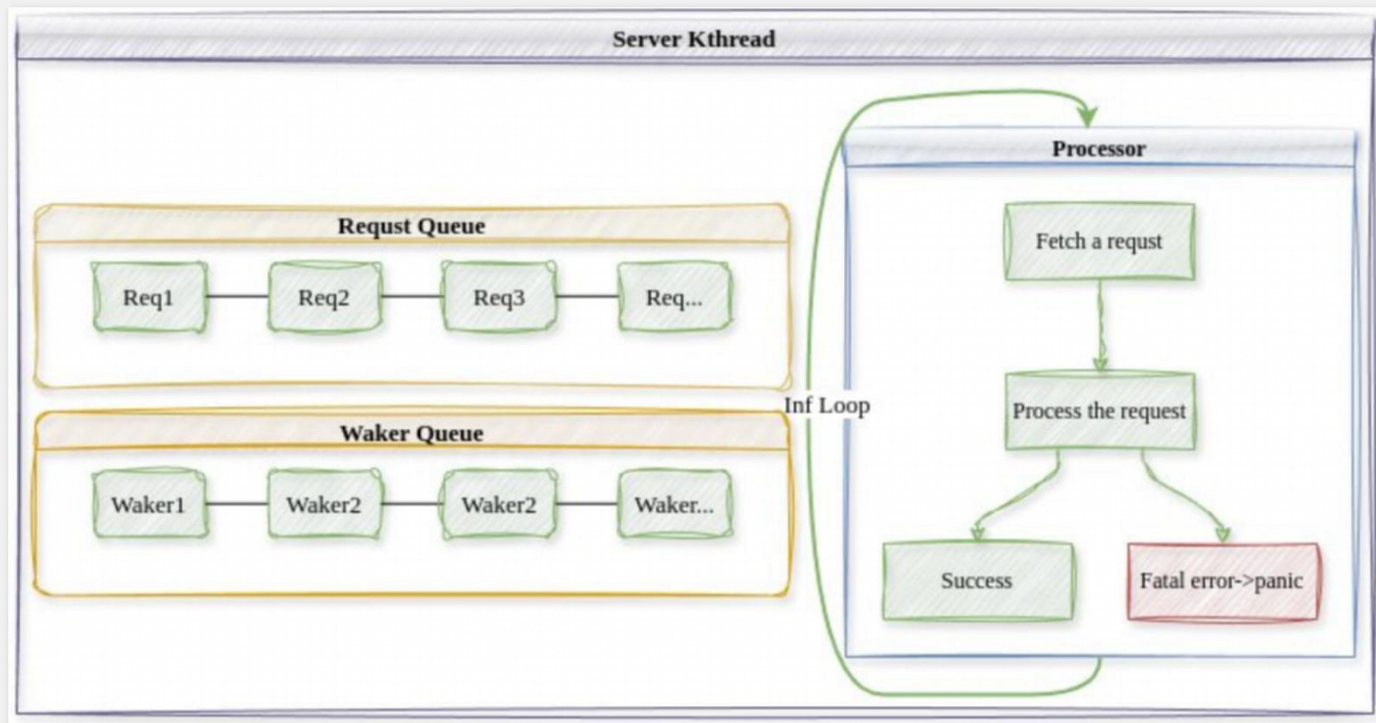
可靠性增强内核架构设计：多对多线程模型的混合内核

- 内核服务线程提供非核心服务（如文件系统、驱动程序等）
- 协程执行线程执行所有用户线程



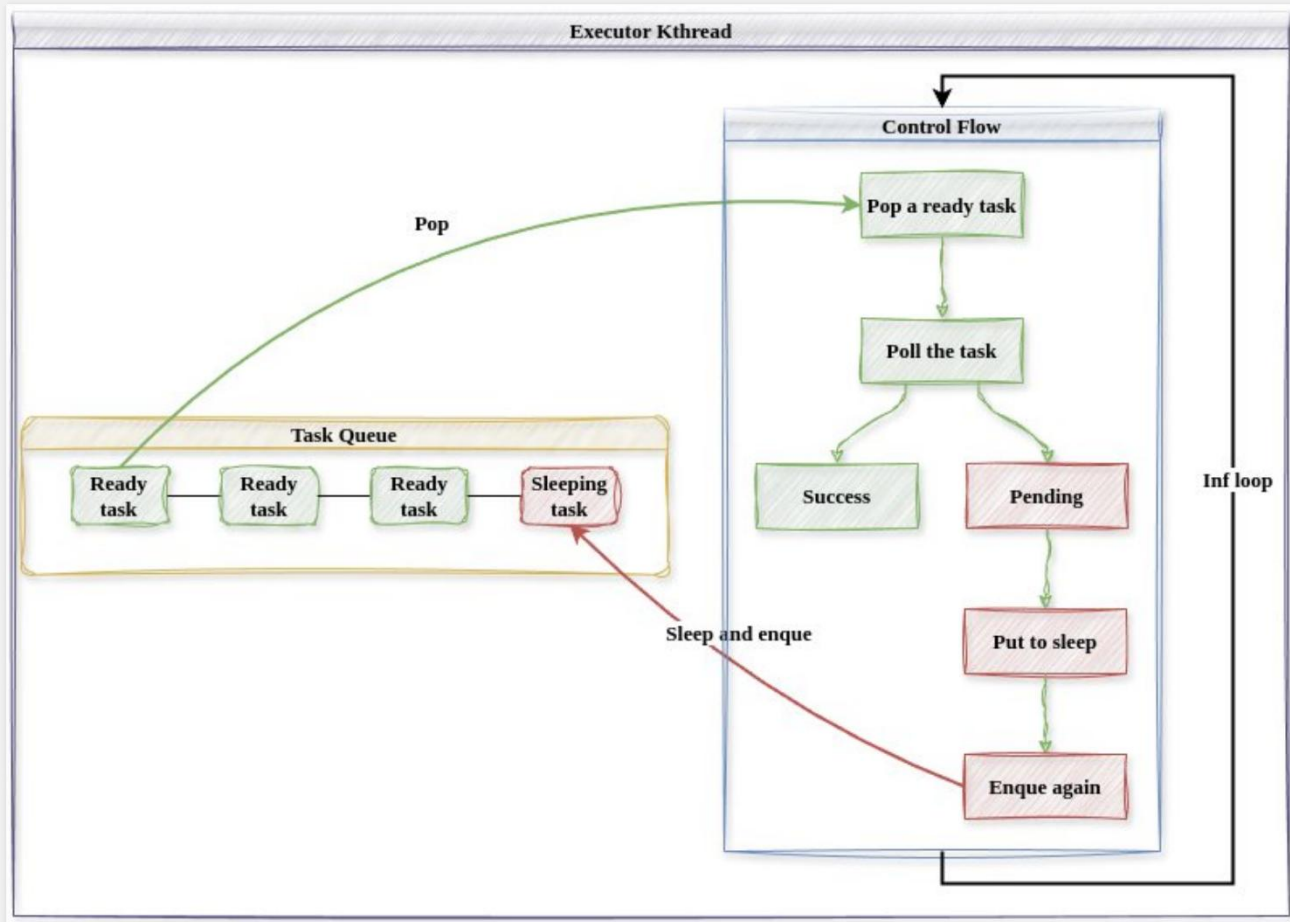
内核服务线程设计

- 提供一种特定的内核服务
- 拥有**独立的内核栈和控制流**
- 崩溃时**不影响其他内核线程和内核**，可尝试重启

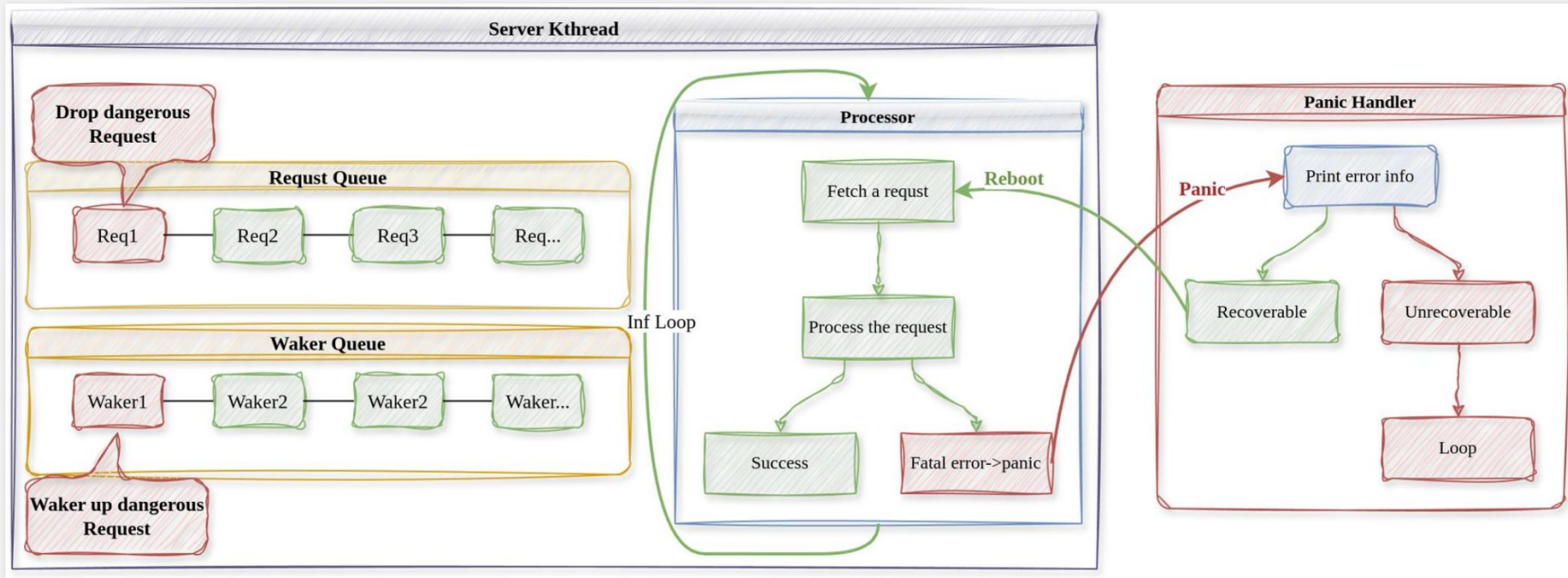


协程执行器线程设计

- 以Rust无栈异步协程代替传统内核线程
- 降低上下文切换开销
- 节省内存



内核服务线程故障恢复



- 内核服务线程故障时，其丢弃出错请求，内核重置内核栈和控制流。
- 任何内核服务线程故障不会影响内核和其余内核服务线程。



03

用户程序支持

musl-libc和C语言用户程序

- 由于直接支持Linux系统调用，只要符合Linux用户栈调用约定和musl线程库实现约定，就可以完成对musl的支持。
- 实际测试时，在Linux宿主机上编译好C程序确保可正常运行，再将elf文件打包到Rust内核中测试运行。

Busybox和sqlite等较大程序

- 使用strace命令查看elf文件使用的系统调用，再在内核中逐个实现。
- 完成busybox支持后，即可直接使用其中自带的ash作为内核的shell程序。
- 目前仍在调试支持中。



04

项目总结

项目总结

- 本项目实现了一个多对多线程模型的混合内核框架，并实现了宏内核与混合内核两种内核架构。
- 完成了对musl-libc和简单c语言用户程序的初步支持。
- 由于目前支持的设备驱动程序数量少，应用程序生态不足，难以说明混合内核对于内核可靠性提升的作用，尚未进行系统的测试分析。

欢迎各位老师批评指正
THANKS

